A Benchmark of Post-Hoc Matching Frameworks for Keyword Spotting Over ASR

Sam Sikora • CMPE 401

Abstract

While generalized Automatic Speech Recognition (ASR) models like OpenAI's Whisper offer state-of-the-art transcription, they underperform in dedicated Keyword Spotting (KWS) tasks due to over-contextualization and vocabulary overshadowing. To mitigate these issues without requiring the resource-intensive training of specialized KWS models, this study proposes and evaluates five post-hoc phonetic matching frameworks applied directly to Whisper's text output. By converting transcriptions into ARPAbet phoneme sequences, the frameworks apply distance metrics of varying complexity: linear search, Levenshtein distance, Euclidean phonetic space distance, and confusion-weighted metric. Of which, the normalized Levenshtein yielded the highest overall performance ($F1$ = 0.933). Ultimately, this research demonstrates that applying tunable, post-hoc phonetic matching can improve KWS accuracy over standard generalized ASR, provided the chosen distance metric fundamentally aligns with the underlying model's actual error behavior.

Introduction

The task of automatic speech recognition (ASR) is a well-trodden field characterized by a wide array of publicly accessible models. Open-source frameworks such as OpenAI's Whisper [1] or NVIDIA's Canary offer state-of-the-art accuracy and robust performance. In contrast, the task of keyword spotting (KWS) cannot typically rely on these generalized solutions; instead, it requires specialized, lightweight models explicitly trained to detect a predefined set of target words [2]. Often, developers with limited resources or without a strong machine learning background will simply deploy a full-sentence ASR model and linearly search the text output for keywords. However, this approach is a poor fit for dedicated keyword detection. Two main issues arise with this workaround:

- **Over-contextualization:** Keywords are isolated utterances, while ASR models often utilize the sentence context along with raw audio to predict the spoken word. This feature risks a spoken keyword being context shifted into another word.
- **Vocabulary Overshadowing:** ASR models are trained to recognize a complete dictionary worth of words, while keyword detection looks to search for a small subset of words. Such errors may occur where a keyword is substituted for a phonetically similar, higher-frequency word from the model's vast dictionary (e.g. misinterpreting the keyword 'porcupine' as 'poor combine').

The result of these issues is that ASR models severely underfit their KWS use case, thus, these systems are seen as unresponsive.

In this work, we propose a post-hoc phonetic matching framework that sits on top of a general-purpose ASR model. Rather than searching for exact string matches in the transcript, each output word is compared against target keywords in phonetic space using ARPAbet phoneme sequences produced from the CMU Pronouncing Dictionary [3]. We implement and evaluate five such detectors of increasing sophistication, ranging from Levenshtein distance to a data-driven confusion-weighted metric derived from Whisper transcription errors.

Related Work

Historically, the field of KWS had been driven by the demand for low-latency, highly accurate keyword detection in computationally constrained environments. These desires originated from need for keyword activation for voice assistants to constantly be running and the ability to be deployed on any hardware. This drove the development of specialized, small-footprint models. A foundational example of this approach is the Deep Keyword Spotting (Deep KWS) framework proposed by Chen et al [2]. However, while highly effective for their respective use cases, such specialized models require dedicated keyword-specific training data and custom architectures, motivating the need for zero-shot, post-hoc phonetic matching atop existing, generalized ASR systems.

A prominent example of a generalized ASR is Whisper [1], a robust, sequence-to-sequence Transformer model trained on 680,000 hours of weakly supervised, multilingual audio data. Unlike traditional models that require dataset-specific fine-tuning, Whisper demonstrates exceptional zero-shot transfer and approaches human-level accuracy across a wide distribution of noisy and diverse environments. Because of its out-of-the-box acoustic representation and transcription robustness, we utilize Whisper as the foundational ASR model for our research. However, as a generalized decoder designed for continuous speech, Whisper heavily relies on long-range text context to resolve acoustic ambiguities. This design makes it inherently susceptible to the over-contextualization and vocabulary overshadowing issues that plague naive ASR-based keyword detection. By layering our proposed phonetic detection frameworks over Whisper's output, we leverage the acoustic robustness of large-scale weak

supervision while re-constraining the search space to accurately isolate predefined keywords.

Phonetic similarity provides a structured way to compare words by their sounds rather than their spelling. The most foundational method to compare two sequences is the Levenshtein edit distance [4], which counts the minimum number of insertions, deletions, and substitutions required to transform one sequence into another. However, there is a key limitation to this method in that it ascribes equal cost to each phonetic substitution. In other words, replacing similar sounding phonetics (/i/ in *sheep* vs /ɪ/ in *ship*) is treated the same as completely dissimilar phonemes. Microsoft's PhoneticMatching library [5] addresses this by mapping each ARPAbet phoneme to a three-dimensional articulatory feature vector, a concept grounded in frameworks such as PatPho [6]. Substitution costs then become Euclidean distances in this feature space, so phonetically similar sounds result in a low cost and phonetically different sounds a high cost. Building upon these metrics, our work both utilizes the Levenshtein and PhoneticMatching library to evaluate ASR transcribe words and keywords in phonetic space, and additionally, create a data-driven confusion metric derived directly from Whisper's actual transcription behaviors.

Methods

To address the limitations of standard ASR for keyword spotting, we developed five different detection pipelines that processes the textual output of the Whisper model. These detection models progress from exact string matching to sophisticated data driven phonetic comparisons.

- **Linear Search:** At the most basic level, linear search serves as our control. It will perform an exact word match between the transcription and the target keyword list. If a match occurs, a detection is recorded. This method serves to analyze the base rate of the Whisper model.
- **Phonetic Levenshtein:** Each transcribed word is converted into its ARPAbet phoneme sequence utilizing the CMU Pronouncing Dictionary. We then calculate the Levenshtein edit distance between the phoneme sequence of the transcribed word and the keyword. A match is registered if the distance is less than or equal to an integer threshold, k. $k = 0$: Requires an identical phoneme sequence. $k = 1$: Permits a single phoneme insertion, deletion, or substitution. $k \geq 2$: Higher thresholds allows more capacity to more detect more words but comes at a cost of higher false-positive rates.
- **Normalize Phonetic Levenshtein:** A significant flaw with counting integer Levenstein distance is its failure to differentiate short keywords. For example, even at threshold $k = 2$, a keyword like "beam" which is composed of 3 phonemes will match to almost anything. To mitigate these false positives, we implemented a normalized edit distance metric. The integer distance is divided by the total number of phonemes in the target keyword, and evaluated against a fractional threshold, r. For example, $r = 0.33$ would translate to at most one-third of the keyword's phoneme may differ. This normalization makes the threshold length-invariant.
- **Phonetic Feature Distance:** The previous approaches treat all phonetic substitutions as equal cost, which does not reflect real world differences between phonetic similarities. To address this, we implement the phonetic distance metric from Microsoft's PhoneticMatching library. Each ARPAbet phoneme is mapped to a three-dimensional articulatory feature vector. Consonants are encoded as (phonation, place of articulation, manner of articulation). Vowels are instead encoded as (0.1, backness, height). The substitution cost between two phonemes is then the Euclidean distance between their vectors. Insertion and deletion costs are phonetic-dependent: vowels carry a cost of 0.5 while consonants carry 0.25, as omitting a vowel is more significant than omitting a consonant.
- **Confusion-Weighted Phonetic Distance:** The Microsoft metric encodes hand-crafted articulatory theory. While this may be perfect for real world similarities, it does not reflect how Whisper specifically fails. Our final detector replaces fixed substitution costs with a derived phoneme confusion matrix built from Whisper's transcription errors on the Google Speech Commands (GSC) training set [7]. For each training clip, Whisper's transcription is aligned against the ground-truth phoneme sequence, and every (reference, hypothesis) phoneme pair is recorded. The substitution cost between phonemes p and q is defined as $1 - P(q \mid p)$, where $P(q \mid p)$ is the probability that Whisper transcribes phoneme p as q. Under this method, a commonly confused pair costs nearly zero, and a never-observed substitution costs 1.0. Insertion and deletion costs remain uniform at 1.0. This yields a detector uniquely calibrated to Whisper's own error distribution rather than idealized theory.

All these detection methods were then benchmark on the Google Speech Commands dataset. This benchmarking was performed by first having Whisper transcribe the command voice clips in the dataset. Out of the thirty keywords in the dataset, twenty were selected as true keywords, and ten were selected as negatives (or words that should not trigger detection). Then using the training split of the GSC, the thresholds for each detector were tuned by maximized macro-averaged F1 across all 20 keywords. The validation split was then transcribed under four conditions: clean audio and Gaussian white noise at signal-to-noise ratio (SNR) 20, 10, and 0 dB. For each condition and detector, per-keyword precision, recall, and F1 were recorded, along with macro-averaged scores.

Results

Detector	Threshold	Precision	Recall	F1
Norm. Phonetic Levenshtein	0.4	0.992	**0.887**	**0.933**
Phonetic Feature Distance	0.2	0.992	0.880	0.929
Confusion-Weighted Phonetic Distance	0.25	**0.994**	0.866	0.922
Linear Search	--	**0.994**	0.866	0.922
Phonetic Levenshtein	1	0.950	0.898	0.918

Table 1: *Macro-averaged scores for each detector's performance on the GSC validation dataset.*

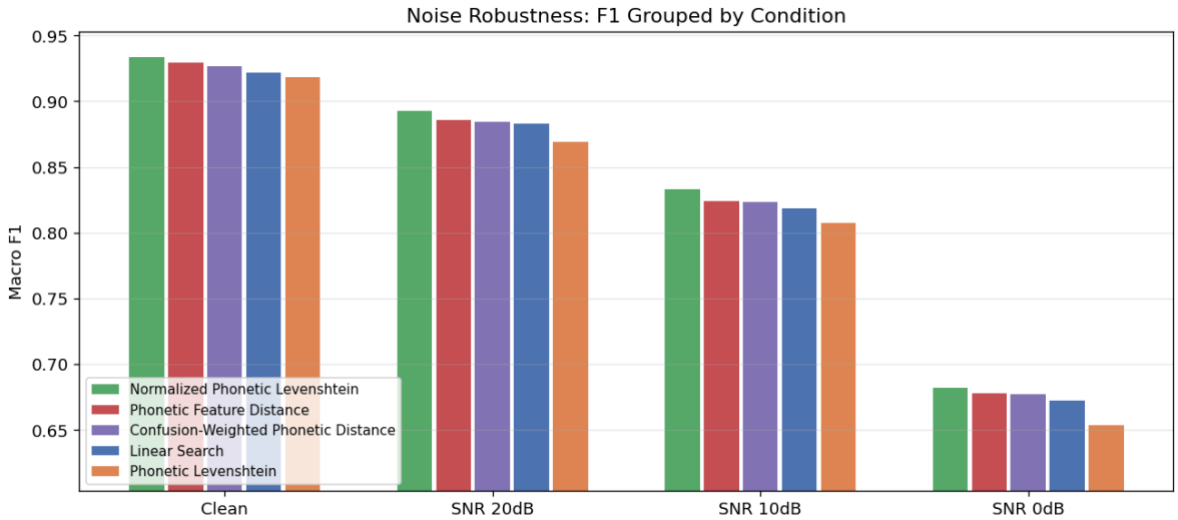

Figure 1: *Macro-averaged F1 scores by noise level for each detector on the GSC validation dataset.*

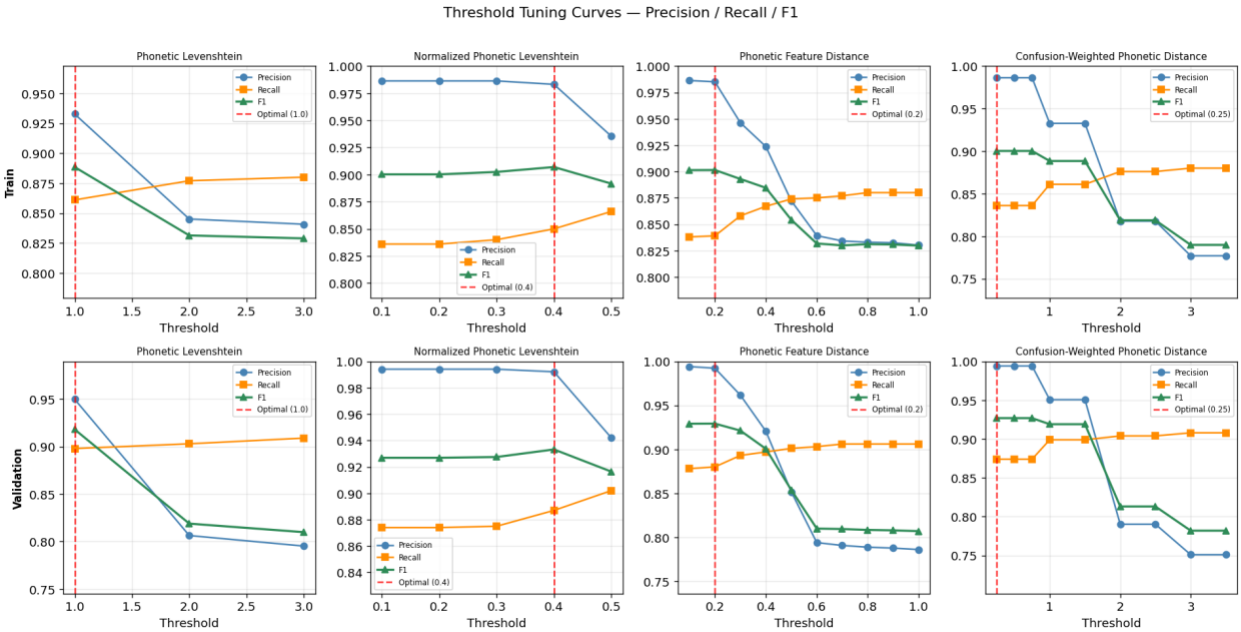

Figure 2: *Precision, recall, and F1 of thresholder detectors over training and validation splits.*

Conclusion

Across all noise levels, a clear ordering of detectors arises. As the noise level increased, the ordering of the detectors

held. Of which, the most optimal F1 detector is the normalized phonetic Levenshtein distance. This was surprising, as this detector was the simplest feasible detector to analyze.

The likely reason for this detector to succeed compared to the more complex detectors is a mismatched assumption of the nature of errors in Whisper. For example, in the keyword 'four', there were three classes of errors:

- A substitution of a single phonetic. This error we would expect catch especially if the phonetics were similar (such as 'four' to 'poor' or 'more').
- Complete mistranslations into other words ('four' as 'bower', 'huh', or 'follow').
- Dropped phonemes ("four" into "or").

Of these three error classes, the complex phonetic detectors were equipped to catch only one, whereas the Normalized Levenshtein Distance successfully captured two. This is because the of the complex detectors ascribed a higher cost to a deletion of a phonetic over a substitution error. The motivation for these different error costs came from the PatPho dataset that seek to mimic human auditory processing. However, because of the strict thresholds that these detectors held, a single deletion cost was too high and would never be classify as a keyword which led to these detectors missing an entire class of error types. Yet, Whisper due to the nature of audio clips or the black box nature of the model, would often make these deletion mistakes. Consequently, a metric that ascribes high cost to deletions is ill-suited for this detection task, as it fundamentally misaligns with the actual error distribution produced by Whisper. This also highlights why the Confusion-Weighted Phonetic Distance detector, trained to conform to Whisper's errors, failed. The detector trained weighted errors for substitution and had a fixed deletion error cost; therefore, it behaved just as the Phonetic Feature distance did.

The simple, discrete Phonetic Levenshtein distance had the worst performance (macro $F1 = 0.918$), as expected given the outlined high false positive rate on the short keywords that compose the GSC dataset. The Linear Search, Confusion-Weighted Phonetic Distance, and Phonetic Feature Distance all exhibited similar performance, clustering tightly between $F1 = 0.922$ and 0.929 — a spread of less than one percentage point. This is likely since the positive and negative keywords in the GSC dataset are composed of one to three phonemes; to avoid false positives on the negative keywords, the models were forced to drive their threshold down. This effectively collapses these detectors towards Linear Search behavior.

The threshold tuning curves in Figure 2 make this tradeoff explicit. As the threshold is relaxed, recall rises monotonically, but precision falls sharply; a pattern seen both in the training and validation splits. These curves represent a tunable parameter for the ASR over the KWS problem. A deployment prioritizing sensitivity over precision could shift the threshold rightward at an acceptable precision cost, and the continuous-valued detectors (Normalized Phonetic Levenshtein, Phonetic Feature Distance, Confusion-Weighted) offer finer control over this tradeoff than the integer-valued Phonetic Levenshtein. While this behavior may be too relaxed for a use-case like voice assistant keyword activation ("Hey Google" or "Hey Siri"), this would be justified when a user is directly interacting with a system such as actively using a voice-controlled software such as playing a voice-controlled video game.

This report and data suggest that applying static detectors over ASR models may lead to a slight improvement in F1 scores in the KWS problem area. This report suggests that any detectors created must align with the way that the ASR model makes errors to be successful. One area to explore further is how these detectors operate with data that incorporates keywords within normal speech. These detectors, due to their nature, may produce many false positives when not interacting with pure keyword data. It may be for these detectors to be helpful acoustically unique keywords need to be choosen.

Works Cited

- [1] A. Radford *et al*, "Robust Speech Recognition via Large-Scale Weak Supervision," 2022.
- [2] G. Chen, C. Parada and G. Heigold, "Small-footprint keyword spotting using deep neural networks," *2014 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Florence, Italy, 2014, pp. 4087-4091, doi: 10.1109/ICASSP.2014.6854370.
- [3] R. L. Weide, "The CMU Pronouncing Dictionary," Carnegie Mellon University, 1998. [Online]. Available: http://www.speech.cs.cmu.edu/cgi-bin/cmudict
- [4] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, Feb. 1966.
- [5] Microsoft Corporation, "PhoneticMatching," GitHub repository, 2021. [Online]. Available: https://github.com/microsoft/PhoneticMatching
- [6] P. Li and B. MacWhinney, "PatPho: c," *Behavior Research Methods, Instruments, & Computers*, vol. 34, (3), pp. 408-415, 2002.
- [7] P. Warden, "Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition," *arXiv preprint arXiv:1804.03209*, 2018. Available: https://arxiv.org/abs/1804.03209.